Day 5 — 피드백 생성 API + LLM 결과 DB 저장

> 작성 중 (2026-07-01). 세션 끝날 때마다 채워 넣는 중.

📌 나중에 다시 설명 들을 것 (지금은 넘어감)

`json.load` vs `json.loads` 차이 ⏳ 나중에 자세히

- **한 줄 직관:** `loads` = "load **s**tring" → 문자열(string)을 읽을 때. s가 문자열 표시.
- `json.load(파일)` : **파일**을 열어서 읽을 때 (파일 객체를 넘김)
- `json.loads(문자열)` : **문자열**을 읽을 때 (그냥 텍스트를 넘김)
- 우리 상황: `result`는 LLM이 돌려준 **문자열**이니까 → `json.loads(result)` 를 씀.
- 하는 일: `'{"next_goal": "..."}'` (문자열) → `{"next_goal": "..."}` (파이썬 딕셔너리)로 바꿔줌. → 그래야 `data["next_goal"]` 처럼 값을 꺼낼 수 있음.
- ❓ 왜 문자열/파일을 구분하는지, 반대(딕셔너리→문자열)는 `json.dumps`인 것 → **다음 복습 때 다시 설명**.

오늘 배운 핵심 개념 (세션 1 완료 — Day 5 ③)

1. LLM 결과를 DB에 저장하는 흐름

```
data = json.loads(result)          # 문자열 → 딕셔너리
new_feedback = Feedback(           # 딕셔너리 값으로 "선반 규격 상자" 만들기
    child_id=child.id,
    lesson_record_id=record.id,
    next_goal=data["next_goal"],
    teacher_feedback=data["teacher_feedback"],
    parent_feedback=data["parent_feedback"],
)
db.add(new_feedback); db.commit(); db.refresh(new_feedback)   # 저장 3종 세트
return new_feedback
```

- `data["key"]`의 key는 딕셔너리에 **실제로 있는 이름과 글자까지 똑같아야** 함 (틀리면 `KeyError`). key 이름은 프롬프트에서 지정한 JSON 형식(`next_goal` 등) 그대로.

2. `FeedbackResponse` 스키마 (돌려줄 데이터 모양)

- 기준 = models.py의 `Feedback` 테이블 칸들. 돌려줄 것: id/child_id/lesson_record_id/next_goal/teacher/parent/created_at.
- `class Config: from_attributes = True` 필요 → **DB 객체(.점으로 읽음)를 Pydantic이 읽을 수 있게 허락**. 없으면 에러. (Pydantic v1 `orm_mode` → v2 `from_attributes`, Day 3에서 만난 거)
- **들여쓰기 규칙:** 왼쪽 끝 = 독립 클래스(형제) / 안으로 들여쓰면 = 바깥 것의 부품(자식). `Config`는 Response 안(자식), 두 스키마는 서로 남남(형제).

3. `List[...]` 응답

- `.all()`은 여러 개(리스트) → `response_model=List[FeedbackResponse]`로 감싸야 함. `from typing import List` 필요.

4. 404 예외처리 `HTTPException`

```
from fastapi import HTTPException
if record is None:                  # .first()는 못 찾으면 None
    raise HTTPException(status_code=404, detail="...")
if not feedbacks:                   # 빈 리스트 []는 not이 True
    raise HTTPException(status_code=404, detail="...")
```

- `raise` = 에러를 일부러 발생. `404` = 표준 "없음" 신호. 없는 ID로 크래시(500) 나기 전에 깔끔히 막음.

5. Swagger 200 vs 422 헷갈림 (해결)

- 위쪽 **"Server response"의 Code** = 진짜 결과 / 아래쪽 **"Responses" 표** = 그냥 "이럴 수도 있어요" 문서.
- path에 값 받는 API엔 FastAPI가 자동으로 422 설명을 붙임 → 실제로 난 에러 아님.